DEAD DROP

Forensics Challenge Write-Up

Category: Forensics
Subcategory: File Analysis
Difficulty: Easy
Challenge File: evidence.raw
Flag Format: OBV{...}

Flag

OBV{D34D_DR0P_UNDER_7H3_V4ULT}

Challenge Description

A suspicious raw file was recovered from a compromised system. The file contains a hidden dead-drop message. The objective is to inspect the artifact, identify the useful forensic clue, extract the encoded payload, and decode it to recover the flag.

The only clue provided to the player is:

```
None
OBV{...}
```

Objective

Analyze evidence.raw and recover the hidden flag.

1. Identify the File

Start by determining what kind of file was recovered:

```
Shell
file evidence.raw
```

Expected output:

```
None
evidence.raw: data
```

Why?

The file command identifies the file type. Since the artifact is reported only as generic data, it suggests that normal file-format metadata is not immediately available. This is a reason to inspect the raw contents.

2. Search for Human-Readable Data

Use strings to extract printable text:

```shell
strings -a evidence.raw
```

The output contains forensic-looking records such as:

```none
[FORENSIC_RECORD]
CASE_ID=DD-047
STATUS=ARCHIVED
ANALYST=UNKNOWN
RECOVERY=PARTIAL
NOTE=DEAD DROP LOCATION COMPROMISED
```

Later, the player discovers:

```
None
--- VAULT TELEMETRY ---
DEAD_DROP=ACTIVE
LOCATION=UNDER
PAYLOAD=T0JWe0QzNERfRFIwUF9VTkRFUl83SDNfVjRVTFR9
--- END TELEMETRY ---
```

Why?

strings extracts readable character sequences from otherwise raw binary data. The VAULT TELEMETRY block is the important forensic artifact.

3. Locate the Payload

Instead of manually searching through the complete strings output, filter for the payload marker:

```
Shell
strings -a evidence.raw | grep 'PAYLOAD='
```

Expected output:

```
None
PAYLOAD=T0JWe0QzNERfRFIwUF9VTkRFUl83SDNfVjRVTFR9
```

Why?

grep searches the strings output for the exact PAYLOAD= marker. This isolates the encoded value from the surrounding forensic noise.

4. Extract and Decode the Payload

The payload looks like Base64 because it uses the Base64 character set and has the expected encoded structure. Extract the value after PAYLOAD= and decode it:

```
Shell
strings -a evidence.raw | grep 'PAYLOAD=' | cut -d= -f2 |
base64 -d
```

Explanation:

. **strings -a evidence.raw:** Extracts printable strings from the raw artifact.

. **grep 'PAYLOAD=':** Selects the line containing the hidden payload.

. **cut -d= -f2:** Splits the line at '=' and selects the value after PAYLOAD=.

. **base64 -d:** Decodes the Base64 data back into plaintext.

5. Recover the Flag

The decoded result and recovered flag is:

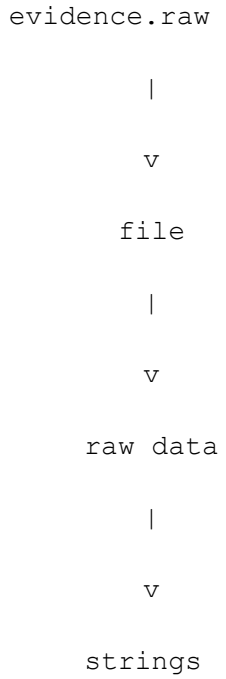
```
None
OBV{D34D_DR0P_UNDER_7H3_V4ULT}
```

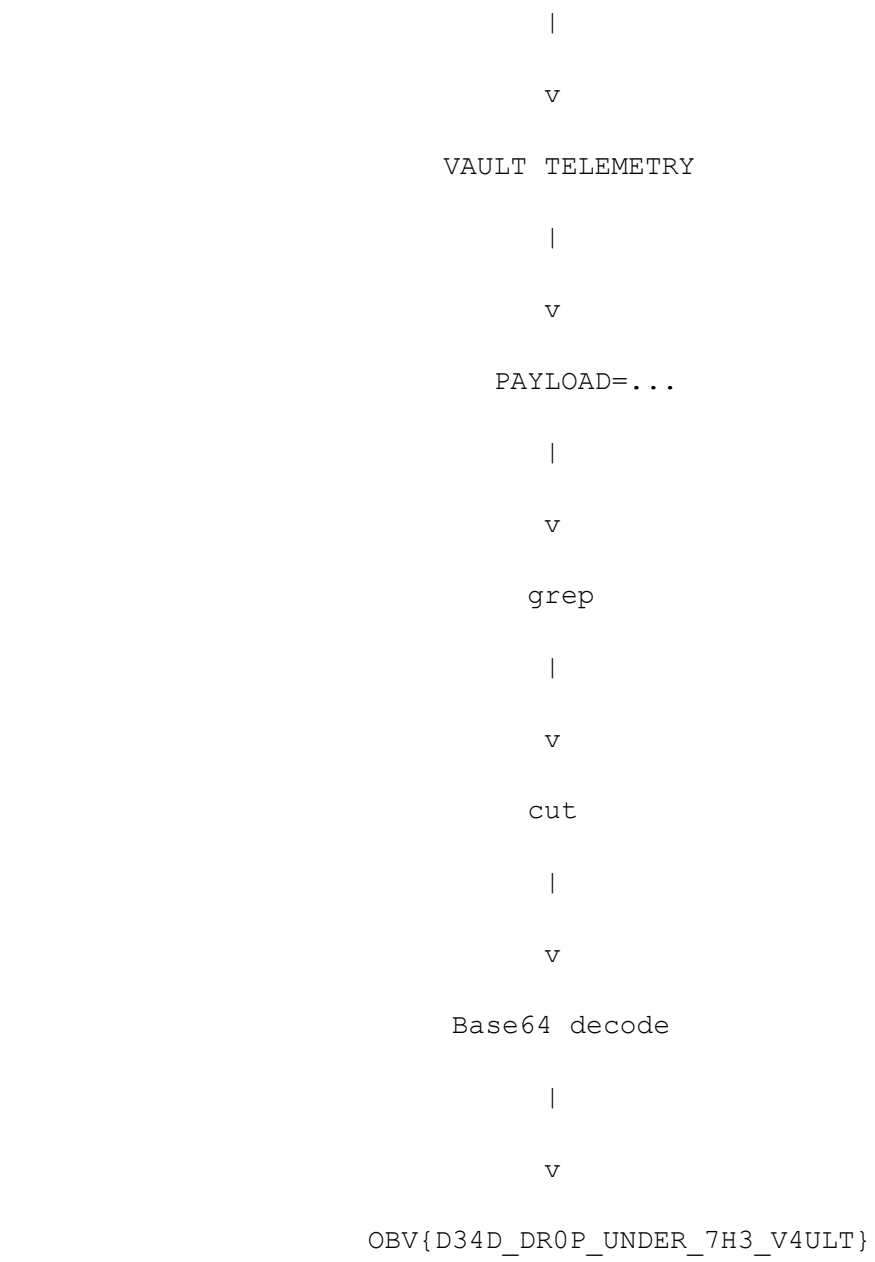
Complete Solve

The complete intended command sequence is:

```shell
file evidence.raw
strings -a evidence.raw
strings -a evidence.raw | grep 'PAYLOAD='
strings -a evidence.raw | grep 'PAYLOAD=' | cut -d= -f2 |
base64 -d
```

Solve Flow




VAULT TELEMETRY
PAYLOAD=...
grep
cut
Base64 decode
OBV{D34D_DR0P_UNDER_7H3_V4ULT}